



Compiladores

Aula 3: Análise léxica

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Tokens, lexemas, padrões e atributos
- Tabela de símbolos
- Erros léxicos

Análise léxica

- A análise léxica é a primeira etapa de um compilador, sendo responsável pela identificação dos tokens.

programa-
fonte

caracteres



ANALISADOR
LÉXICO



tokens



parser

- O analisador léxico é usualmente representado pelo **scanner**, que é capaz de ler o arquivo do código-fonte e identificar seus elementos.

Análise léxica

- Como transformar uma sequência de caracteres em unidades que o **parser** consiga compreender?
- O **parser** é a segunda etapa do Compilador, o qual realiza a análise semântica do código.
- O scanner responde
“que unidade léxica é esta?”;
- o parser responde
“como essas unidades se organizam?”.

Token, padrão, lexema e atributo

Token

Nome abstrato usado pelo parser.

Ex.: IDENTIFIER, INTEGER, PLUS

Lexema

Sequência concreta de caracteres que casa com um padrão.

Ex.: total, _x, contador2

```
int valor = 42;
```

Token	Lexema
INT (palavra reservada)	int
IDENTIFIER	valor
ASSIGN	=
INTEGER (literal)	42
SEMICOLON (delimitador)	;

No MINIC, operadores e delimitadores normalmente não precisam de atributo.

Padrão

Descrição da forma que um lexema pode assumir.

Ex.: [A-Za-z_][A-Za-z0-9_]*

Atributo

Informação associada ao token.

Ex.: valor 42 ou referência na tabela

Token, padrão, lexema e atributo

- O scanner classifica ocorrências concretas em categorias que o parser reconhece.

Palavras reservadas	int · if · while · return	literal da palavra
Identificadores	soma · _valor · contador2	[A-Za-z_][A-Za-z0-9_]*
Literais	0 · 42 · 3.14 · "texto"	regras numéricas e de delimitação
Operadores	+ · - · * · / · == · <=	testar os mais longos primeiro
Delimitadores	() { } [] ; ,	um token por símbolo
Comentários	// ... · /* ... */	descartar antes de emitir tokens

IMPORTANTE: A ordem das regras importa, por exemplo: == deve ser reconhecido antes de =.

Token, padrão, lexema e atributo

- O scanner deve olhar um ou mais caracteres à frente para decidir onde o lexema termina.
- Este método é denominado “*Lookahead*”, o lookahead decide se o próximo caractere pertence ao mesmo token.

Exemplos de lookahead

= ASSIGN vs == EQUAL

< LESS vs <= LESS_EQUAL

Token, padrão, lexema e atributo

- O scanner lê os caracteres do código-fonte através de um “**ponteiro de leitura**”.
- Exemplo:

[p][o][s][i][t][i][o][n]



ponteiro de leitura

Token, padrão, lexema e atributo

- Exemplo

`position = initial + rate * 60;`

lexemas



- Tokens

<ID, position> ASSIGN <ID, initial> PLUS <ID, rate> STAR <INT, 60> SEMICOLON

IMPORTANTE: os espaços são descartados, mas linha e coluna devem permanecer disponíveis para diagnósticos.

Token, padrão, lexema e atributo

- Tratamento de espaços e comentários.

```
int x = 10;    // comentário descartado
/* bloco descartado */  x = x + 1;
```

Descartar

Espaços, tabulações e comentários não geram tokens.

Atualizar

Linha e coluna devem avançar durante toda a leitura.

Preservar

A posição de cada token apoia as mensagens de erro.

O scanner transforma a entrada, mas também prepara o diagnóstico.
Descartar texto não significa perder informação útil para diagnosticar erros.

Tabela de símbolos

- Identificadores conectam a sequência de tokens às informações semânticas do programa.



nome	categoria	tipo e dimensões	escopo e posição
position	IDENTIFIER	INTEGER	FUNCTION main, ROW 2, COL 5

Erros léxicos

- Os erros léxicos são identificados pela falta de aderência ao padrão esperado.

Erro	Exemplo	Identificação
Símbolo não permitido	<code>int x = 10 @ 2;</code>	linha 1, coluna 12
String não terminada	<code>print("MINIC);</code>	esperava aspas de fechamento
Literal malformado	<code>int x = 3.14.5;</code>	formato numérico inválido
Comentário sem fechamento	<code>/* início...</code>	esperava */

Scanner do projeto

- Escreva o código Python referente ao parser do projeto MINIC.

Código de teste:

```
int main() {
int total = 2 + 3 * 4;
// mostra o resultado
print(total); return 0; }
```

01 Liste os tokens

nome, lexema, linha, coluna e atributo

02 Introduza erros

símbolo inválido e cadeia não terminada

03 Verifique o diagnóstico

a mensagem permite localizar e corrigir?

Checklist da Etapa 1: palavras reservadas · operadores longos · comentários · posições · erros · testes

Scanner do projeto

- Chamada do scanner:

```
python scanner.py file.c
```

Saídas como texto no terminal para esta primeira versão